NLP / LLM PROJECT DOCUMENTATION

DevMind

PDF question answering with retrieval-augmented generation

STUDENT	STUDENT
Hafiz Muhammad Hashir Roll number B22110006042	**Wasi Uz Zafar Khan** Roll number B22110006124

1. Project overview

DevMind is a document question-answering application. A user uploads a PDF, asks a question, and receives an answer grounded in that file. The matching passages are shown beside the chat. It is a full-stack NLP project built around retrieval-augmented generation (RAG). A large language model writes the answer. Search, storage, reranking, and evaluation run on the machine that hosts the application.

2. Features

- Upload PDF, TXT, or Markdown from the Library page.
- RAG mode answers from retrieved passages and shows citations.
- Baseline mode asks the same model with no documents, for comparison.
- Chat history supports follow-up questions, rename, pin, and delete.
- Search can be limited to one document.
- Retrieved passages are reranked locally before the model call.
- The prompt is trimmed so the model request stays small.
- The Compare page reports recall and precision without calling the model.

3. API key

The project uses one API key.

Name	Purpose
`GEMINI_API_KEY`	Google AI Studio key, stored in `backend/.env`

The server reads the key at startup. The key is not written in source code, and `.env` is not committed to git. There is no OpenAI key, no login provider, and no database password.

4. How the LLM works

Gemini is the only large language model. It is called once per question.

1. **Ingest.** Text is extracted from the file and split into passages of about 900 characters, with 150 characters of overlap. Each passage is embedded and the vectors are saved.

2. **Retrieve.** The question is embedded, together with the last two user messages so follow-ups stay on topic. Passages are ranked with cosine similarity, then reordered by a local word-overlap score. The best few passages are kept.
3. **Generate.** Those passages, a system instruction, and recent chat turns are sent to Gemini. The model must answer from that context and cite sources as `[Source 1]`.
4. **Baseline.** The same Gemini call is made with no passages attached.

Chat models are tried in this order if one is busy: `gemini-3.5-flash`, `gemini-3.1-flash-lite`, `gemini-flash-lite-latest`, `gemini-3.8-flash`.

Embeddings use `gemini-embedding-001`. If that call fails, a local MiniLM model (`Xenova/all-MiniLM-L6-v2`) creates the vectors. Passages and questions must use the same embedding model, so files should be uploaded again after an embedding change.

Step	Implementation
Chunking	LangChain Recursive Character TextSplitter
Embeddings	`backend/src/rag/embeddings.js`
Vector store	JSON files and cosine similarity in `backend/src/rag/store.js`
Rerank and filter	`backend/src/rag/rerank.js`
Context limit	`backend/src/rag/context.js`
Generation	`backend/src/rag/gemini.js`
Recall and precision	`backend/src/rag/eval.js` (no Gemini call)

5. Tech stack

Layer	Technologies
Frontend	React 19, Vite, Tailwind CSS 4
Backend	Node.js, Express, Multer, pdf-parse
LLM	Google Gemini through `@google/genai`
NLP	LangChain text splitter, local MiniLM embeddings

6. Setup

Node.js 20 or newer is required. Create a key at `https://aistudio.google.com/api-keys` and save it as `backend/.env`:

```
GEMINI_API_KEY=your_key_here
PORT=4000
```

Start the API on port 4000:

```
cd devmind/backend
npm install
npm start
```

Start the interface on port 5174:

```
cd devmind/frontend
npm install
npm run dev
```

Open `http://localhost:5174`.

7. Demo flow

1. **Library** – upload a PDF and wait until it is indexed.
2. **Chat** – leave the mode on RAG and ask a question about that file. Citations appear on the right.
3. Switch to **Baseline** and ask the same question to show the difference.
4. Open an older chat in the sidebar and ask a follow-up.
5. **Compare** – click *Check retrieval*. Recall is how often the right passage was in the top results. Precision is how many of those results were actually relevant. This check uses no Gemini tokens.

8. HTTP API

Method	Path	Purpose
`GET`	`/api/health`	Document and passage counts
`GET`	`/api/documents`	List indexed files
`POST`	`/api/documents`	Upload a file (multipart field `file`)
`DELETE`	`/api/documents/:id`	Remove a file and its vectors
`GET`	`/api/conversations`	List chats
`POST`	`/api/conversations`	Start a chat
`GET`	`/api/conversations/:id`	Load one chat
`PATCH`	`/api/conversations/:id`	Rename or pin a chat
`DELETE`	`/api/conversations/:id`	Delete a chat
`POST`	`/api/conversations/:id/messages`	Ask in a chat. Body: `question`, `mode`, `docId`
`POST`	`/api/eval`	Local recall and precision
`POST`	`/api/optimize`	Choose top-K from those local scores
`POST`	`/api/ask`	One-off question, with no saved chat

`mode` is either `rag` or `baseline`.

9. Project structure

```
devmind/
├── backend/
│   ├── .env
│   ├── data/
│   │   └── documents, vectors, chats
│   └── src/
│       ├── index.js
│       └── rag/
│           ├── pipeline.js
│           ├── gemini.js
│           ├── embeddings.js
│           ├── store.js
│           ├── rerank.js
│           ├── context.js
│           ├── eval.js
│           ├── conversations.js
│           └── retrievalSettings.js
└── frontend/
    └── src/
        ├── App.jsx
        ├── api.js
        └── components/
```

10. Production build

```
cd devmind/frontend
npm run build
npm run preview
```